ORDER 59 — Theorie des Empreintes : formalisme, architecture et resultats etablis

Reconstruction experimentale ORDER 59

20 septembre 2026

Contents

1 Mandat et principes de conception **2**

2 Empreintes, fibres et quotient **2**

3 Raffinement et fermeture **2**

4 Seuils et loi du minimum-double **2**

5 Deficit : transformer l'echec en information **3**

6 Construction active de la representation **3**

7 Contraintes construites par l'agent **3**

8 Prediction et monde interne **3**

9 Imagination comme hypothese executable **4**

10 Synthese de programmes de raisonnement **4**

11 EA, latent canonique, reflexion et circulation **4**

12 Memoire et apprentissage cumulatif **4**

13 Capacites et transfert **4**

14 Anti-triche et integration **4**

15 Resultats experimentaux etablis **5**

16 Conclusions strictement supportees **5**

17 References internes **5**

Abstract

ORDER 59 transforme le noyau en agent de construction active de representation. L'entree n'est plus un probleme deja formalise : l'agent recoit une observation brute, un objectif public et un environnement d'interaction. Les empreintes, fibres, deficits, sondes, contraintes, predictions, hypotheses d'imagination et programmes sont alors construits et utilises pendant la resolution. Ce document decrit les parties formalisees, implementees et observees ; les tentatives, echecs et variantes sont separes dans le document ESSAYE.

1 Mandat et principes de conception

Le mandat O59 impose quatre non-regressions. Premierement, ne pas remplacer la Theorie des Empreintes par un solveur standard. Deuxiemement, ne pas donner au modele une representation cible deja preconstruite. Troisiemement, ne pas transformer les difficultes computationnelles en impossibilites scientifiques. Quatriemement, toute brique majeure doit avoir un effet causal mesurable ou une preuve mathematique identifiable.

Le noyau ne recoit donc ni `Problem`, ni solution, ni liste de coups chess legaux, ni evaluation chess. Les environnements restent des mecanismes d'evaluation externes et leurs seules informations publiques sont celles de l'observation et du but.

2 Empreintes, fibres et quotient

Pour un ensemble d'observations X et des sondes $P = (p_1, \dots, p_k)$:

$$I_P(x) = (p_1(x), \dots, p_k(x)), \qquad x \sim_P y \iff I_P(x) = I_P(y).$$

La fibre est $[x]_P = \{y : I_P(y) = I_P(x)\}$. Le module `core.py` construit directement les partitions de fibres.

La suffisance d'une empreinte relativement a une cible A est :

$$\mathrm{Suff}(I_P, A) \iff I_P(x) = I_P(y) \Rightarrow A(x) = A(y).$$

Le programme `factorize` realise la construction de $\bar{A}$ lorsque la cible est constante sur les fibres :

$$A = \bar{A} \circ I_P.$$

Inversement, la coexistence d'une collision $I_P(x) = I_P(y)$ avec $A(x) \neq A(y)$ est un contre-exemple d'insuffisance ; aucune recherche supplementaire fonctionnant sur la meme information ne peut reconstruire simultanement les deux valeurs.

3 Raffinement et fermeture

Une sonde supplementaire g produit $I' = (I, g)$ et garantit :

$$[x]_{I'} \subseteq [x]_I.$$

Cette propriete est verifiee par `refinement_partition`. La fermeture empirique collecte les sondes dont l'ajout laisse la partition inchangee. Ainsi, une sonde peut etre informative sans etre necessaire a chaque niveau de resolution.

4 Seuils et loi du minimum-double

Pour une suite de sondes ordonnee, le seuil de paire est le plus grand niveau jusqu'auquel les descriptions restent identiques. Le test O59 verifie sur support fini :

$$\Theta(x, z) \geq \min\{\Theta(x, y), \Theta(y, z)\}.$$

La distance discrete

$$d(x,y) = 0 \ (x = y), \qquad d(x,y) = e^{-\Theta(x,y)} \ (x \neq y)$$

est ainsi ultrametrique sur le support du test lorsque la loi est satisfaite. Ces proprietes sont des proprietes du formalisme ; la generalisation a un support infini demande d'autres hypotheses.

5 Deficit : transformer l'echec en information

L'agent porte le vecteur :

$$D = (D_{obs}, D_{cap}, D_{res}, D_{search}, D_{transfer}, D_{constraint}).$$

D_{obs} est augmente lorsque des observations partageant une empreinte ont des consequences observees divergentes. $D_{constraint}$ refl ete des interactions rejetees que les contraintes courantes n'expliquent pas. D_{res} est derive de la consommation reelle du budget. Les autres composantes représentent des besoins de ressources ou de consolidation, pas des impossibilites.

Le meta-controle choisit ensuite entre PROBE, CONSTRAINT, SYNTHESIZE, TRANSFER, RESOURCE, IMAGINE, SEARCH et VERIFY. Cette decision apparait dans les traces de chaque episode.

6 Construction active de la representation

Le percepteur transforme des observations arbitraires en objets observables sans reconnaitre a priori une famille de probleme. Une banque generique de sondes fournit des fonctions de type, de longueur, de chemin, de cle, d'index, de bucket, de signe et de hash tronque. La banque est ensuite reduite par le feedback des episodes.

La boucle O59 appelle la construction de representation a l'episode initial puis periodiquement. `EAAttention` calcule une signature predictive de consequence et le `NeuralProbeController` produit un second classement appris en ligne. Le choix actif est donc effectue a partir des observations et des consequences reelles, et non d'un label de benchmark.

7 Contraintes construites par l'agent

Les contraintes sont des objets avec domaine, relation, predicate, confiance et provenance. Les primitives de composition implementent :

$$C_1 \wedge C_2, \qquad C_1 \vee C_2,$$

et le raffinement comme conjonction. La relaxation utilise les deux entrées dans une disjonction ; cette implementation conserve le comportement corrige d'O55/O56 et empeche la regression ou une seule contrainte serait effectivement active.

L'induction part des affectations effectivement observees et des interactions acceptees/rejetees. Il n'existe pas de branche `if benchmark == ...` dans le noyau.

8 Prediction et monde interne

Le modele predictif conserve d'abord un support empirique :

$$\widehat{P}(s' \mid s, a) = \frac{N(s,a,s')}{\sum_u N(s,a,u)}.$$

En parallele, apres accumulation suffisante, un modele dynamique lineaire est ajuste sur un vecteur de caracteristiques derivees de l'observation. La prediction porte une confiance et un support de transitions connus. La composition multi-horizon est :

$$\hat{s}_{t+1} = W(\phi(s_t), a_t), \quad \hat{s}_{t+h} = W(\hat{s}_{t+h-1}, a_{t+h-1}).$$

La version O59 distingue ainsi explicitement memorisation de transitions et extrapolation interne.

9 Imagination comme hypothese executable

Chaque hypothese est un objet structurel type avec provenance. Les familles actuellement proposees sont probe, constraint, program, action sequence et representation. La branche retenue est envoyee dans le modele predictif interne :

$$H \xrightarrow{simulate_W} \widehat{y}_H \xrightarrow{compare} \Delta_H.$$

Le resultat de simulation influence les propositions d'action. Cette causalite est tracee par `imagination_execution`, avec le nombre de pas simules et la confiance moyenne de prediction. Aucune observation future de l'environnement n'est lue pendant cette simulation.

10 Synthese de programmes de raisonnement

Le moteur de synthese O59 ne se limite plus a quatre heuristiques de variables. Sa grammaire composee contient notamment Observe, Probe, Refine, ConstructConstraint, Predict, Imagine, Compare, Select, Rank, Branch, Loop, Store, Retrieve, Act, Verify, Backtrack et Replan. Les programmes sont des AST types et sont executes par `ProgramVM`, sans `eval` ni `exec`.

Le score d'un candidat est obtenu par execution de la VM dans des contextes tires des transitions reelles. L'enumeration est finie et budgetee ; une completude universelle n'est pas revendiquee.

11 EA, latent canonique, reflexion et circulation

`geometry.py` contient les primitives de seuils, fermeture, circulation et reflexion. `latent.py` construit des cellules canoniques a partir de signatures de sondes et enregistre des connexions de voisinage. Le latent n'est donc pas une coordonnee Euclidienne imposee a l'avance ; il represente la classe issue de la description construite.

12 Memoire et apprentissage cumulatif

La memoire episodique conserve les actions, recompenses, deficits et hash d'observation. Les programmes synthetises peuvent etre stockes et recuperes a l'episode suivant. La memoire semantique gere invalidation et remplacement avec provenance. La campagne repetitive de sequence n'a toutefois pas produit de gain de score mesurable entre le deuxieme episode avec memoire et le controle sans memoire ; aucun gain cumulatif n'est donc declare.

13 Capacites et transfert

Une capacite candidate porte programme, signature et evidence. La promotion compare une condition OFF et une condition ON et refuse la promotion lorsque le gain causal n'est pas positif.

Le transfert decouvre des correspondances par signature structurelle. La certification appelle ensuite un runner cible. Le test archive retrouve une correspondance complete entre trois invariants et la certification retourne VERIFIED. Cette experience etablit le chemin de certification, pas une generalite de transfert sur tous les domaines.

14 Anti-triche et integration

L'audit statique interdit les appels eval/exec/os.system, les dispatch explicites sur benchmark/task/family et les motifs de tables de solutions. Les environnements sont separes de l'agent par une API generique observe/act. Chess est transmis comme grille 8x8 ; sa legalite reste une propriete du moteur d'evaluation externe.

Le noyau ne contient donc pas de solveur CSP dedie aux benchmarks O55/O56/O58. Les algorithmes de recherche generiques peuvent apparaitre dans certaines actions, mais ils sont commandes par les objets appris et restent instrumentes par la representation, le deficit, le meta-controle et la memoire.

15 Resultats experimentaux etablis

La campagne finale archivee comprend : HiddenPermutation, Sequence, Maze, VisualPattern, Chess mate-in-one et Chess open. Tous sont partiels sous le budget final. Les traces montrent cependant que la boucle de representation et les hypotheses d'imagination sont executees pendant les episodes ; les programmes sont synthetises et evalues par VM.

Le resultat principal n'est donc pas un taux de reussite universel, mais la verification que le chemin causal complet est effectivement parcouru et instrumente sur des observations brutes. Les performances detaillees, commandes et ablations sont dans le rapport d'experience.

16 Conclusions strictement supportees

O59 montre une implementation executable de la boucle informationnelle centrale : observation, empreinte, collision, deficit, sonde, raffinement, prediction, imagination, programme, action et retour. Les proprietes mathematiques centrales sont testees sur support fini. Les campagnes multi-domaines demontrent l'absence d'une conversion automatique en solveur standard, mais les resultats de resolution restent partiels dans les budgets de cette passe. Les questions de generalisation ouverte, de world model general, de synthese illimitee et d'apprentissage cumulatif restent des objets experimentaux et ne sont pas declares resolus.

17 References internes

[O5] `DDD_Base_theorie_des_empreintes_complete_order_5.tex`.

[O55] `model_order_55.zip` et rapport d'experience O55. [O56] `model_order_56.zip` et rapport d'experience O56. [O58] `theorie_empreintes_order_58_bundle.zip` et `model_order_58.zip`. [EA] documents EA du corpus, notamment la formalisation du tour, de la reflexion et de la bifurcation. [v0] pipeline complet et formalisme prediction du corpus.